



projeto 4

Programação de Computadores - Experiência Prática 4



2 Identificação do aluno

projeto 4
Programação de Computadores - Experiência
Prática 4

SS

**Samuel dos Santos Escoval CRUZ_EAD_Cst em Análise e Desenvolvimento de
Sistemas_1A_20261**



Estrutura de dados

A estrutura de dados solicitada no enunciado consiste em uma lista de dicionários, na qual cada estudante é representado por um dicionário contendo, no mínimo, os atributos "nome" e "notas". Um exemplo dessa abordagem seria: `estudantes = [ { "nome": "Alana", "notas": [4.0, 5.0] }, { "nome": "Samuel", "notas": [8.0, 7.0] } ]` Essa solução atende ao requisito proposto, pois permite armazenar e consultar os dados dos estudantes de forma organizada e flexível. Durante o desenvolvimento do sistema, inicialmente considerei utilizar listas simples, porém percebi que seria difícil identificar o significado de cada posição armazenada. Em seguida, avaliei a utilização de dicionários, que oferecem maior clareza por meio de chaves nomeadas. À medida que novas funcionalidades foram sendo adicionadas ao projeto, como cálculo de médias, recuperação, verificação de aprovação, classificação por faixa etária, controle de repetência e previsão de conclusão do curso, a quantidade de atributos e comportamentos associados a cada estudante aumentou significativamente. Por esse motivo, optei por utilizar uma classe chamada `Aluno`, aplicando conceitos de Programação Orientada a Objetos. Dessa forma, cada estudante passou a ser representado como um objeto contendo seus próprios atributos e métodos, facilitando a organização do código, a reutilização das funcionalidades e a manutenção do sistema. Exemplo da estrutura adotada: `class Aluno: def __init__(self, nome, idade, nota1, nota2, ano_entrada, nota_recuperacao=None): self.nome = nome self.idade = idade self.nota1 = nota1 self.nota2 = nota2 self.ano_entrada = ano_entrada self.nota_recuperacao = nota_recuperacao` Posteriormente, os objetos da classe foram armazenados em uma lista denominada `turma`: `turma = [ Aluno("Alana", 20, 4, 5, 2020, 8), Aluno("Samuel", 17, 8, 7, 2024) ]` Essa abordagem mantém a facilidade de armazenamento em lista, mas adiciona uma melhor organização dos dados e das regras de negócio do sistema.



Funções de cálculo e regras institucionais

No enunciado, a proposta inicial era criar funções independentes como ``calcular_media(notas)`` e ``verificar_aprovacao(media, media_minima=7.0)``, seguindo o princípio da responsabilidade única. Essa abordagem funciona bem porque cada função realiza apenas uma tarefa: uma calcula a média e a outra verifica a aprovação. No meu desenvolvimento, mantive essa mesma ideia de modularidade, porém optei por aplicar a lógica dentro de uma classe chamada ``Aluno``. Essa decisão foi tomada porque o sistema passou a trabalhar com mais informações além das notas, como idade, ano de entrada, recuperação, faixa etária, repetência e previsão de conclusão do curso. Com isso, a classe facilitou a organização dos dados e das regras relacionadas a cada estudante. A função de média foi implementada como um método da classe: `python def calcular_media(self): """Calcula a média inicial do aluno.""" return (self.nota1 + self.nota2) / 2` Nesse caso, em vez de receber uma lista de notas como parâmetro, o método utiliza diretamente os atributos do próprio objeto, ``self.nota1`` e ``self.nota2``. Também foi criada a função ``calcular_media_final``, responsável por considerar a nota de recuperação quando ela existir: `python def calcular_media_final(self): """Calcula a média final considerando recuperação, se houver.""" media = self.calcular_media() if media >= 7: return media if self.nota_recuperacao is None: return media return (media + self.nota_recuperacao) / 2` Essa função permite tratar três situações diferentes: aluno aprovado direto, aluno ainda em recuperação e aluno que já possui nota de recuperação. Para verificar a situação acadêmica do aluno, criei o método ``verificar_situacao``: `python def verificar_situacao(self): """Verifica a situação acadêmica do aluno.""" media = self.calcular_media() media_final = self.calcular_media_final() if media >= 7: return "Aprovado" if self.nota_recuperacao is None: return "Em recuperação" if media_final >= 7: return "Aprovado após recuperação" return "Reprovado"` Dessa forma, embora o enunciado apresente como base a função ``verificar_aprovacao(media, media_minima=7.0)``, no meu projeto a lógica foi expandida para contemplar também a recuperação. A regra principal de aprovação foi mantida, considerando média maior ou igual a 7, mas acrescentei uma verificação intermediária para alunos que ainda precisam realizar recuperação. Caso fosse necessário seguir exatamente o formato solicitado no enunciado, a lógica poderia ser escrita assim: `python def calcular_media(notas): return sum(notas) / len(notas) def verificar_aprovacao(media, media_minima=7.0): if media >= media_minima: return "Aprovado" return "Reprovado"` No entanto, como o sistema desenvolvido ficou mais completo, optei por organizar essa lógica dentro da classe ``Aluno``, mantendo o princípio de responsabilidade única por método e deixando o código mais coerente com os dados de cada estudante.



Geração de relatórios

No enunciado, a função solicitada foi `gerar\_relatorio(alunos)`, que receberia uma estrutura completa de estudantes e exibiria no terminal o nome, a média e a situação de aprovação de cada aluno. No meu projeto, como optei por organizar o sistema usando a classe `Aluno`, adaptei essa ideia para um método chamado `mostrar\_relatorio(self)`. Esse método pertence a cada objeto aluno e utiliza os próprios atributos da classe para gerar o relatório individual. Além das informações básicas solicitadas, como nome, média e situação, também incluí dados complementares, como idade, faixa etária, nota de recuperação, repetência, tempo total do curso e ano previsto de conclusão. Essa escolha foi feita porque o sistema desenvolvido possui regras adicionais relacionadas à recuperação e à conclusão do curso. Código utilizado: ``python def mostrar_relatorio(self): """Mostra o relatório completo do aluno.""" media = self.calcular_media() media_final = self.calcular_media_final() situacao = self.verificar_situacao() faixa = self.faixa_etaria() repetiu = self.verificar_repetencia() tempo_total, ano_conclusao = self.conclusao_curso() print("\n==== RELATÓRIO DO ALUNO =====") print(f"Aluno: {self.nome}") print(f"Idade: {self.idade}") print(f"Faixa etária: {faixa}") print(f"Nota 1: {self.nota1}") print(f"Nota 2: {self.nota2}") print(f"Média inicial: {media:.2f}") print(f"Nota recuperação: {self.nota_recuperacao}") print(f"Média final: {media_final:.2f}") print(f"Situação: {situacao}") print(f"Repetiu: {'Sim' if repetiu else 'Não'}") print(f"Tempo total do curso: {tempo_total} anos") print(f"Ano previsto de conclusão: {ano_conclusao}") `` Para apresentar o relatório de todos os alunos da turma, utilizei um laço de repetição percorrendo a lista `turma`: ``python for aluno in turma: aluno.mostrar_relatorio() `` Dessa forma, cada aluno gera seu próprio relatório, mantendo o código organizado e coerente com a estrutura orientada a objetos adotada no projeto.



Docstrings e padrões

Para atender às boas práticas de documentação e às recomendações da PEP 8, adicionei docstrings às funções do sistema. As docstrings têm como objetivo descrever a finalidade da função, seus argumentos de entrada (Args) e os valores retornados (Returns), facilitando a manutenção e a compreensão do código por outros desenvolvedores. Além disso, todos os métodos, funções e variáveis foram nomeados utilizando o padrão snake_case, conforme recomendado pela PEP 8. Alguns exemplos são:

``calcular_media`, `verificar_situacao`, `nota_recuperacao`, `ano_entrada` e `gerar_relatorio`.` Abaixo estão duas funções do sistema documentadas: ```python def calcular_media(self): """ Calcula a média inicial do aluno. Args: Nenhum argumento externo. Utiliza os atributos self.nota1 e self.nota2. Returns: float: Média aritmética das duas notas. """ return (self.nota1 + self.nota2) / 2 ```python def verificar_situacao(self): """ Verifica a situação acadêmica do aluno. Args: Nenhum argumento externo. Returns: str: Situação do aluno, podendo ser: 'Aprovado', 'Em recuperação', 'Aprovado após recuperação' ou 'Reprovado'. """ media = self.calcular_media() media_final = self.calcular_media_final() if media >= 7: return "Aprovado" if self.nota_recuperacao is None: return "Em recuperação" if media_final >= 7: return "Aprovado após recuperação" return "Reprovado" ``` Essas docstrings permitem identificar rapidamente o objetivo de cada função, os dados utilizados e os resultados produzidos, tornando o código mais legível, organizado e alinhado às boas práticas de desenvolvimento de software. ```



Conteúdo do README.md

Sistema de Gerenciamento de Notas de Estudantes ## Descrição Este projeto é um sistema simples de gerenciamento acadêmico desenvolvido em Python. O sistema permite cadastrar estudantes, calcular médias, verificar situação de aprovação, considerar nota de recuperação, classificar faixa etária, verificar repetência, prever o ano de conclusão do curso, pesquisar alunos e gerar estatísticas da turma. O projeto foi estruturado com foco em modularidade, organização do código, uso de docstrings e aplicação de boas práticas de nomenclatura em Python. ## Funcionalidades * Cadastro de alunos * Cálculo da média inicial * Cálculo da média final com recuperação * Verificação da situação acadêmica * Classificação por faixa etária * Verificação de repetência * Previsão do ano de conclusão do curso * Exibição de relatório completo * Pesquisa por nome ou característica * Estatísticas gerais da turma * Validação de notas e idade ## Requisitos Para executar o projeto, é necessário ter o Python instalado na máquina. Versão recomendada: ``bash Python 3.10 ou superior `` ## Estrutura do Projeto Uma estrutura sugerida para o projeto é: ``text projeto\_notas/ | |—— main.py |—— test\_aluno.py |—— README.md `` O arquivo `main.py` contém o código principal do sistema. O arquivo `test\_aluno.py` pode ser utilizado para armazenar os testes unitários. O arquivo `README.md` contém a documentação do projeto. ## Como executar o sistema 1. Abra o terminal ou prompt de comando. 2. Acesse a pasta onde o projeto está salvo: ``bash cd projeto\_notas `` 3. Execute o arquivo principal: ``bash python main.py `` 4. Após executar o programa, será exibido um menu com as seguintes opções: ``text ===== SISTEMA ESCOLAR ===== 1 - Ver alunos 2 - Adicionar aluno 3 - Pesquisar alunos 4 - Estatísticas da turma 5 - Sair `` ## Como usar o menu ### 1 - Ver alunos Exibe o relatório completo de todos os alunos cadastrados, mostrando nome, idade, faixa etária, notas, média inicial, nota de recuperação, média final, situação acadêmica, repetência, tempo total do curso e ano previsto de conclusão. ### 2 - Adicionar aluno Permite cadastrar um novo aluno informando: * Nome * Idade * Nota 1 * Nota 2 * Ano de entrada * Nota de recuperação, caso exista O sistema valida notas entre 0 e 10 e idade entre 1 e 120 anos. ### 3 - Pesquisar alunos Permite pesquisar alunos por nome ou por características, como: ``text aprovados reprovados adultos jovens idosos `` ### 4 - Estatísticas da turma Exibe informações gerais da turma, como: * Total de alunos * Quantidade de aprovados * Quantidade de reprovados * Média geral da turma * Melhor aluno ### 5 - Sair Encerra a execução do programa. ## Como executar os testes Caso o projeto possua um arquivo de testes chamado `test\_aluno.py`, os testes podem ser executados pelo terminal. ### Usando unittest ``bash python -m unittest test\_aluno.py `` ### Exemplo de teste ``python import unittest from main import Aluno class TestAluno(unittest.TestCase): def test_calcular_media(self): aluno = Aluno("Teste", 20, 8, 6, 2024) self.assertEqual(aluno.calcular_media(), 7.0) def test_aluno_aprovado(self): aluno = Aluno("Teste", 20, 8, 7, 2024) self.assertEqual(aluno.verificar_situacao(), "Aprovado") def test_aluno_reprovado(self): aluno = Aluno("Teste", 20, 2, 3, 2024, 4) self.assertEqual(aluno.verificar_situacao(), "Reprovado") if __name__ == "__main__": unittest.main() `` ## Boas práticas aplicadas Durante o desenvolvimento foram aplicadas boas práticas de programação, como: * Uso de nomes em snake_case * Organização do sistema em funções e métodos * Uso de docstrings * Separação de responsabilidades * Validação de entradas * Uso de classe para organizar os dados dos alunos * Uso de lista para armazenar a turma ## Observações Embora o enunciado inicial proponha o uso de lista de dicionários, neste projeto a estrutura foi adaptada para uma lista de objetos da classe `Aluno`. Essa escolha foi feita para organizar melhor os dados e comportamentos relacionados a cada estudante, principalmente porque o sistema passou a incluir regras adicionais, como recuperação, repetência, faixa etária e previsão de conclusão do curso. ## Autor Projeto desenvolvido como atividade prática de programação em Python.



Testes unitários

```
import unittest
def calcular_media(notas):
    if not notas:
        return 0.0
    return sum(notas) / len(notas)
def verificar_aprovacao(media, media_minima=7.0):
    if media >= media_minima:
        return "Aprovado"
    return "Reprovado"
class TestNotas(unittest.TestCase):
    def test_aprovacao_com_media_normal(self):
        notas = [8.0, 7.0, 9.0]
        media = calcular_media(notas)
        self.assertEqual(media, 8.0)
        self.assertEqual(verificar_aprovacao(media), "Aprovado")
    def test_reprovacao_com_media_normal(self):
        notas = [4.0, 5.0]
        media = calcular_media(notas)
        self.assertEqual(media, 4.5)
        self.assertEqual(verificar_aprovacao(media), "Reprovado")
    def test_lista_de_notas_vazia(self):
        notas = []
        media = calcular_media(notas)
        self.assertEqual(media, 0.0)
    def test_media_minima_zero(self):
        media = 0.0
        self.assertEqual(verificar_aprovacao(media, media_minima=0), "Aprovado")
if __name__ == "__main__":
    unittest.main()
```

Reflexão sobre testes

Os testes do sistema foram realizados utilizando a biblioteca `unittest`, permitindo validar automaticamente o comportamento das principais funções responsáveis pelo cálculo de médias e verificação da aprovação dos estudantes. Inicialmente, foram criados testes para situações normais de funcionamento, verificando se alunos com médias superiores à média mínima eram classificados como aprovados e se alunos com médias inferiores eram classificados como reprovados. Além dos casos comuns, também foram testados casos extremos (edge cases). Foi realizada a validação do comportamento da função `calcular\_media()` quando recebe uma lista de notas vazia, garantindo que o sistema não apresente erros de execução e retorne um valor adequado. Também foi testada a função `verificar\_aprovacao()` utilizando o valor zero como média mínima de corte, verificando se a lógica permanece estável mesmo em condições pouco usuais. Essa estratégia permitiu validar tanto os cenários esperados quanto situações limite, aumentando a confiabilidade do sistema e reduzindo a possibilidade de falhas durante sua utilização.

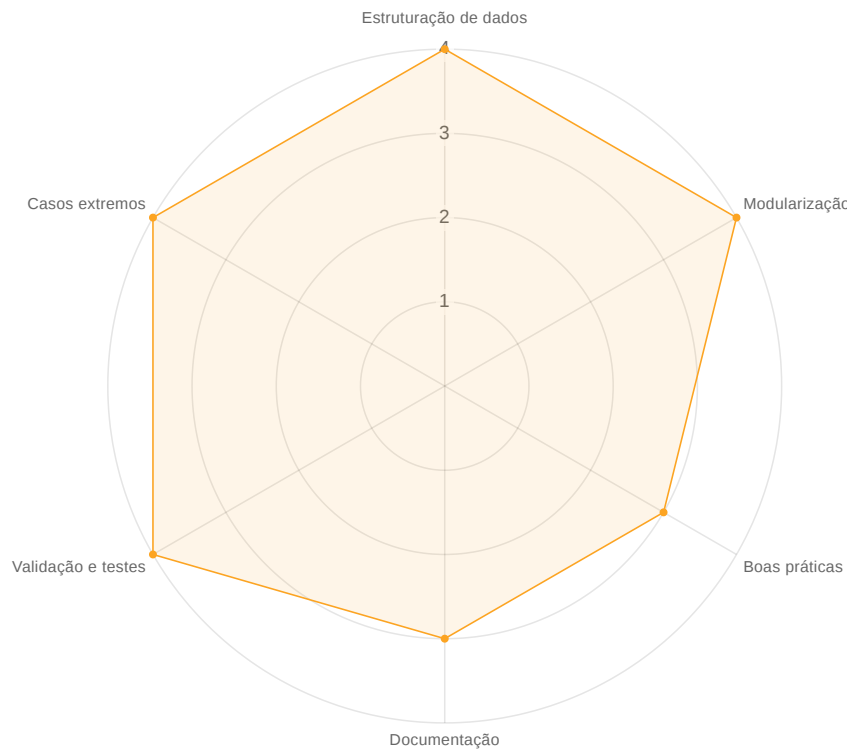
Submissão integrada do sistema

```
# --- gerenciador_notas.py --- ```python def calcular_media(notas): """ Calcula a média aritmética de uma lista de notas. Args: notas (list): Lista de notas do estudante. Returns: float: Média calculada. Retorna 0.0 caso a lista esteja vazia. """ if not notas: return 0.0 return sum(notas) / len(notas) def verificar_aprovacao(media, media_minima=7.0): """ Verifica se o estudante foi aprovado ou reprovado. Args: media (float): Média obtida pelo estudante. media_minima (float): Média mínima necessária para aprovação. Returns: str: 'Aprovado' ou 'Reprovado'. """ if media >= media_minima: return "Aprovado" return "Reprovado" class Aluno: """ Representa um aluno com dados pessoais, notas e informações do curso. """ def __init__(self, nome, idade, nota1, nota2, ano_entrada, nota_recuperacao=None): self.nome = nome self.idade = idade self.nota1 = nota1 self.nota2 = nota2 self.ano_entrada = ano_entrada self.nota_recuperacao = nota_recuperacao def calcular_media(self): """ Calcula a média inicial do aluno. Args: Nenhum argumento externo. Utiliza os atributos self.nota1 e self.nota2. Returns: float: Média aritmética das duas notas. """ return calcular_media([self.nota1, self.nota2]) def calcular_media_final(self): """ Calcula a média final considerando recuperação, se houver. Args: Nenhum argumento externo. Returns: float: Média final do aluno. """ media = self.calcular_media() if media >= 7: return media if self.nota_recuperacao is None: return media return (media + self.nota_recuperacao) / 2 def verificar_situacao(self): """ Verifica a situação acadêmica do aluno. Args: Nenhum argumento externo. Returns: str: Situação do aluno. """ media = self.calcular_media() media_final = self.calcular_media_final() if media >= 7: return "Aprovado" if self.nota_recuperacao is None: return "Em recuperação" if media_final >= 7: return "Aprovado após recuperação" return "Reprovado" def faixa_etaria(self): """ Classifica o aluno por faixa etária. Returns: str: 'Jovem', 'Adulto' ou 'Idoso'. """ if self.idade < 18: return "Jovem" if self.idade < 60: return "Adulto" return "Idoso" def verificar_repetencia(self): """ Verifica se o aluno repetiu. Returns: bool: True se foi reprovado, False caso contrário. """ return self.verificar_situacao() == "Reprovado" def conclusao_curso(self): """ Calcula o tempo total e o ano previsto de conclusão. Returns: tuple: Tempo total do curso e ano previsto de conclusão. """ tempo_total = 2 if self.verificar_repetencia(): tempo_total += 1 ano_conclusao = self.ano_entrada + tempo_total return tempo_total, ano_conclusao def resumo(self): """ Exibe um resumo simples do aluno. Returns: None """ media = self.calcular_media() situacao = self.verificar_situacao() faixa = self.faixa_etaria() print(f"{self.nome} | Média: {media:.2f} | {situacao} | {faixa}") def mostrar_relatorio(self): """ Exibe o relatório completo do aluno. Returns: None """ media = self.calcular_media() media_final = self.calcular_media_final() situacao = self.verificar_situacao() faixa = self.faixa_etaria() repetiu = self.verificar_repetencia() tempo_total, ano_conclusao = self.conclusao_curso() print("\n===== RELATÓRIO DO ALUNO =====") print(f"Aluno: {self.nome}") print(f"Idade: {self.idade}") print(f"Faixa etária: {faixa}") print(f"Nota 1: {self.nota1}") print(f"Nota 2: {self.nota2}") print(f"Média inicial: {media:.2f}") print(f"Nota recuperação: {self.nota_recuperacao}") print(f"Média final: {media_final:.2f}") print(f"Situação: {situacao}") print(f"Repetiu: {'Sim' if repetiu else 'Não'}") print(f"Tempo total do curso: {tempo_total} anos") print(f"Ano previsto de conclusão: {ano_conclusao}") def ler_nota(mensagem): """ Solicita e valida uma nota entre 0 e 10. Args: mensagem (str): Texto exibido ao usuário. Returns: float: Nota válida. """ while True: nota = float(input(mensagem)) if 0 <= nota <= 10: return nota print("Nota inválida. Digite uma nota entre 0 e 10.") def ler_idade(): """ Solicita e valida a idade do aluno. Returns: int: Idade válida entre 1 e 120. """ while True: idade = int(input("Idade: ")) if 0 < idade <= 120: return idade print("Idade inválida. Digite uma idade entre 1 e 120.") def pesquisar_alunos(turma): """ Pesquisa alunos por característica ou nome. Args: turma (list): Lista de objetos Aluno. Returns: None """ filtro = input("Digite uma característica ou nome " "(aprovados, reprovados, adultos, jovens, idosos): ").lower() print("\n===== RESULTADO DA PESQUISA =====") encontrou = False for aluno in turma: situacao = aluno.verificar_situacao().lower() faixa = aluno.faixa_etaria().lower() nome = aluno.nome.lower() if filtro == "aprovados" and "aprovado" in situacao: aluno.resumo() encontrou = True elif filtro == "reprovados" and situacao == "reprovado": aluno.resumo() encontrou = True elif filtro == "adultos" and faixa == "adulto": aluno.resumo() encontrou = True elif filtro == "jovens" and faixa == "jovem": aluno.resumo() encontrou = True elif filtro == "idosos" and faixa == "idoso": aluno.resumo() encontrou = True elif filtro in nome: aluno.resumo() encontrou = True if not encontrou: print("Nenhum aluno encontrado.") def estatisticas_turma(turma): """ Exibe estatísticas gerais da turma. Args: turma (list): Lista de objetos Aluno. Returns: None """ if not turma: print("A turma está vazia.") return total = len(turma) aprovados = 0 reprovados = 0 soma_medias = 0 melhor_aluno = turma[0] for aluno in turma: situacao = aluno.verificar_situacao() media = aluno.calcular_media_final() soma_medias += media if "Aprovado" in situacao: aprovados += 1 if situacao == "Reprovado": reprovados += 1 if media > melhor_aluno.calcular_media_final(): melhor_aluno = aluno media_geral = soma_medias / total print("\n===== ESTATÍSTICAS DA TURMA =====") print(f"Total de alunos: {total}") print(f"Aprovados: {aprovados}") print(f"Reprovados: {reprovados}") print(f"Média geral da turma: {media_geral:.2f}") print(f"Melhor aluno: {melhor_aluno.nome}") def gerar_relatorio(alunos): """ Gera o relatório completo de todos os alunos. Args: alunos (list): Lista de objetos Aluno. Returns: None """ for aluno in alunos: aluno.mostrar_relatorio() turma = [ Aluno("Alana", 20, 4, 5, 2020, 8), Aluno("Samuel", 17, 8, 7, 2024), Aluno("Carlos", 65, 3, 4, 2021, 5), Aluno("Maria", 30, 5, 1, 2026, 9), Aluno("Fernanda", 22, 10, 9, 2025), Aluno("Lucas", 16, 2, 3, 2023, 4), Aluno("Juliana", 45, 6, 6, 2022, 10), ] if __name__ == "__main__": while True:
```



Verificação das atividades realizadas

- Estruturei os dados utilizando listas e dicionários adequadamente conforme a exigência.
- Modulizei o código criando as funções independentes de cálculo e verificação solicitadas.
- Documentei as funções com *docstrings* detalhadas e respeitei os padrões de formatação da PEP 8.
- Escrevi o README.md descrevendo com clareza o propósito do projeto e as suas orientações de uso.
- Criei e executei integralmente os testes de validação, cobrindo inclusive todos os casos extremos indicados.



Reflexão

Considero que tive um bom desempenho durante a realização desta atividade prática. Ao longo do desenvolvimento do sistema, consegui aplicar conceitos importantes de programação, como funções, estruturas de dados, orientação a objetos, validação de entradas, documentação e testes. Um dos meus pontos fortes foi a capacidade de analisar os problemas propostos e buscar soluções próprias para implementá-los. Em alguns momentos, encontrei caminhos diferentes dos sugeridos inicialmente pelo enunciado, principalmente na modelagem dos estudantes, optando pela utilização de uma classe `Aluno` para organizar melhor os atributos e comportamentos do sistema. Essa decisão demonstrou minha capacidade de raciocinar sobre diferentes abordagens de desenvolvimento e adaptar a solução conforme a complexidade do projeto aumentava. Também considero como ponto positivo a evolução percebida em relação aos desafios anteriores. Durante esta atividade consegui compreender melhor conceitos como métodos, classes, listas, funções com responsabilidade única, modularização do código e documentação por meio de docstrings. Além disso, tive contato com a criação de testes unitários, compreendendo a importância da validação automática para garantir a confiabilidade do software. Como oportunidade de melhoria, reconheço que ainda preciso desenvolver mais prática na escrita de código e na utilização de algumas ferramentas e bibliotecas, especialmente relacionadas a testes automatizados e organização profissional de projetos. Em alguns momentos tive dificuldade para compreender determinados requisitos ou para escolher a melhor estrutura de implementação, o que demonstra a necessidade de continuar praticando e aprofundando meus conhecimentos. De forma geral, considero que os aprendizados construídos nesta experiência contribuíram significativamente para meu desenvolvimento profissional. A atividade permitiu consolidar conceitos fundamentais da programação, fortalecer meu raciocínio lógico e ampliar minha compreensão sobre boas práticas de engenharia de software. Esses conhecimentos serão importantes tanto para projetos acadêmicos futuros quanto para minha formação como profissional da área de tecnologia.